RARITONE BACKEND VIRTUAL FASHION PLATFORM

Bharath Kumar | Backend Developer | Raritone Private Limited | 1 Month Internship

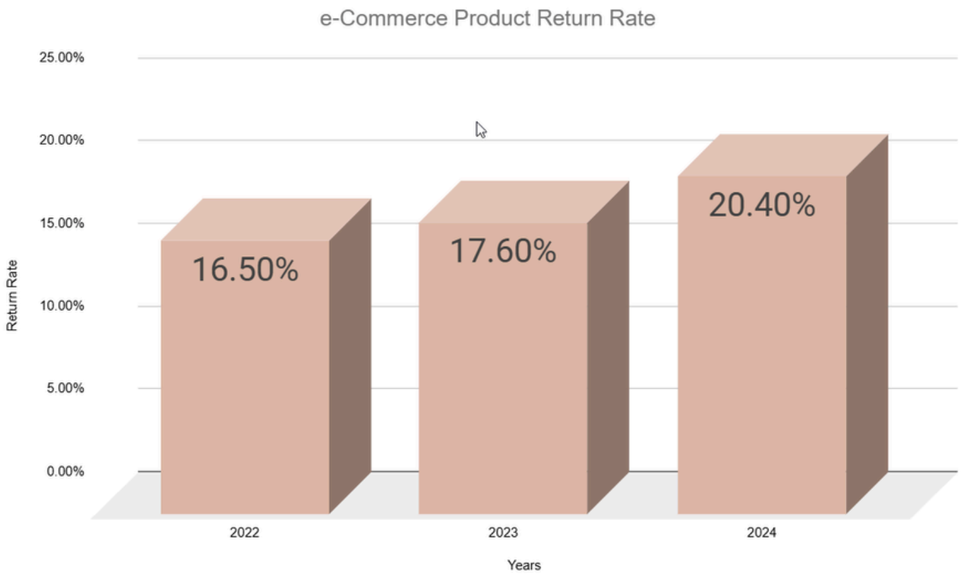
AGENDA

Problem & Objectives
Problem Statement · Objectives · Technology Stack · System Architecture · Database Design

Modules & Implementation
Modules Developed · Security Implementation · Media Processing Workflow · API Highlights · Testing & Results

AI, Scalability & Wrap-Up
AI Integration Planning · Scalability Research · Challenges & Learnings · Team & My Contributions · Conclusion & Future Work

PROBLEM STATEMENT



- Customers cannot visualize outfits before purchase, leading to poor buying decisions and dissatisfaction. Size and fit recommendations are highly inaccurate, resulting in high product return rates of 20–40% — a major operational and financial burden for e-commerce fashion platforms.

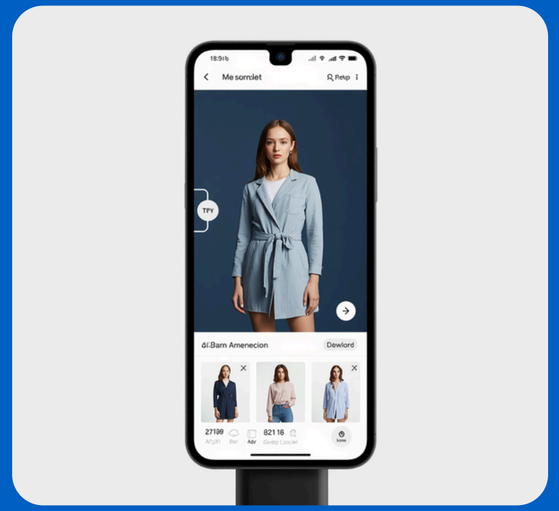
- Existing platforms lack personalization, digital wardrobe management, and virtual try-on features. Users can't digitally manage clothing or receive AI-driven outfit suggestions—a gap Raritone aims to bridge.

OBJECTIVES

- Create secure, scalable RESTful APIs with Node.js and Express.js. Use JWT access tokens (15 min) and refresh tokens (7 days) via Google OAuth 2.0. Implement Role-Based Access Control (USER, ADMIN, SUPER_ADMIN) for all protected routes.

- Manage core modules: users, products, cart, orders, wishlist, wardrobe, and measurements. Store precise body dimensions to enable accurate size recommendations and reduce return rates.

- Create an AI-ready architecture for virtual try-on with real-time updates using Socket.IO, design avatar generation, personalized recommendations, and scalable media processing with Sharp and Cloudinary.



TECHNOLOGY STACK

Backend & Database
Node.js – Runtime environment
Express.js – RESTful API framework
MongoDB – NoSQL database
Mongoose ODM – Schema modeling & queries

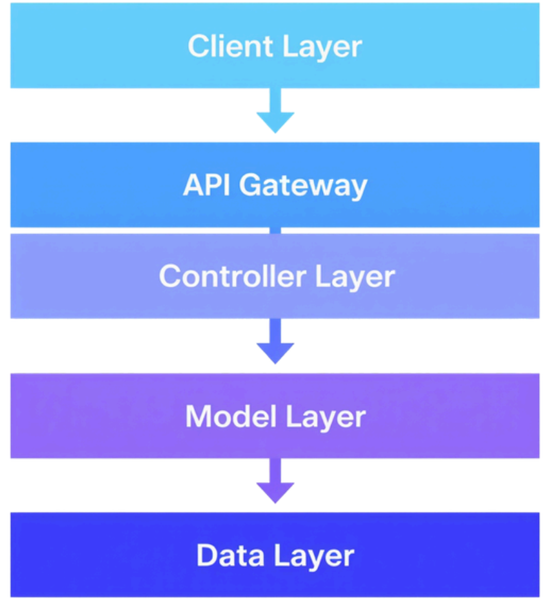
Auth & Security
JWT – Access & refresh tokens
bcrypt – Password hashing
Google OAuth 2.0 – Social login
Helmet – Secure HTTP headers
Joi – Request validation

Media, Real-Time & Utilities
Multer & Sharp – Upload & image processing
Cloudinary – Cloud media storage & CDN
Socket.IO – Real-time try-on updates
Nodemailer – Email notifications
Morgan – HTTP request logging

SYSTEM ARCHITECTURE

- Client Layer
  - Web & Mobile Applications (React / Flutter)
  - API Gateway
  - Express.js Routes — /api/*
- Middleware Layer
  - CORS
  - Helmet
  - Rate Limiting
  - JWT Authentication
  - Joi Validation
- Controller Layer
  - Business Logic — 12+ Modules
    - Auth
    - Product
    - Cart
    - Order
    - Wishlist
    - Wardrobe
- Model Layer
  - Mongoose Schemas & ODM
- Data Layer
  - MongoDB Atlas

Cloudinary CDN



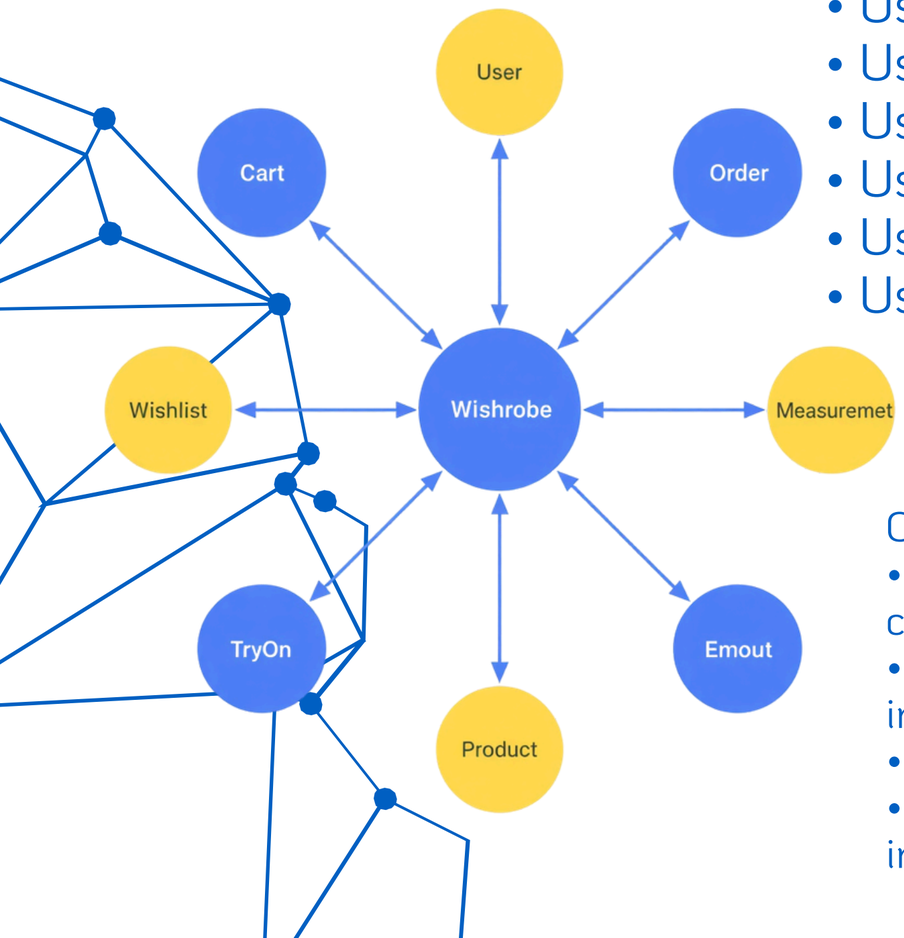
DATABASE DESIGN KEY COLLECTIONS

User: Authentication, roles, reset tokens | Product: Catalog with title, price, images, stock | Cart: One cart per user with product array

Order: Orders with shipping address and status tracking | Wishlist: Saved products per user | Wardrobe: Digital clothing items management

Measurement: Body dimensions (chest, waist, hip, height, shoulder) | TryOn: Original and generated try-on images | ProductMapping: Metadata for AI recommendations

DATABASE RELATIONSHIPS



User References (Mongoose Populate)

- User → Cart: Each user has one cart populated with product array
- User → Order: Order history linked via user reference
- User → Wishlist: Saved products populated with full product details
- User → Wardrobe: Digital clothing items tied to user profile
- User → Measurement: Body dimensions (chest, waist, hip, height, shoulder)
- User → TryOn: Original and AI-generated try-on images stored per user

Cart & Order → Product References

- Cart: References Product documents using .populate('products.productId') to retrieve live catalog data including title, price, and images
- Order: Stores product snapshots (price, details at time of purchase) to preserve order history integrity
- ProductMapping: Linked to Product catalog for AI-ready recommendation metadata
- Example: Cart.findOne({ user }).populate('products.productId') returns full product objects inline

MODULES DEVELOPED (PART 1)

***Authentication Module**

- Signup and login use JWT access tokens (15 min) and refresh tokens (7 days).
- Google OAuth for social login.
- Passwords hashed with bcrypt.
- Password reset via email with Nodemailer.

Role-based access for USER, ADMIN, and SUPER_ADMIN.

Product Module

- Complete CRUD operations for the product catalog. Image upload uses Multer, Sharp (resize to 800px, JPEG 80%), and Cloudinary.
- Offers advanced search and filtering by keyword, category, and price. Removes old images from Cloudinary when updated or deleted.

Cart & Order Modules

- Cart: Add/remove items, update quantities, retrieve per-user cart with populated product data.
-
- Order: Create orders with shipping address, track status workflow — PENDING → SHIPPED → DELIVERED.
-
- Product snapshots stored at order time.

MODULES DEVELOPED (PART 2)

- **Wishlist Module**
  - Add/remove products with duplicate prevention.
  - Retrieves full product details.
  - Prevents adding the same product twice.
- **Wardrobe Module**
  - Add clothing items with metadata and image uploads.

Manage personal digital wardrobe collections.

- **Measurements Module:**
  - Save, update, and delete body dimensions (chest, waist, hip, height, shoulder)
  - Enables accurate size recommendations
- **Try-On Module:**
  - Upload original and generated try-on images
  - Store try-on history per user

Delete functionality for managing sessions

- **Image Module**
  - Generic CRUD operations for all image types:
    - Profile photos
    - Avatars
    - Product images
    - Try-on images
  - Unified image management across all modules
- **Testing**
  - All modules tested via Postman
  - Populated response validation

Protected route checks

SECURITY IMPLEMENTATION

- ***JWT Authentication:***
  - Access tokens: Expire in 15 minutes
  - Refresh tokens: Last 7 days
  - Tokens: Signed with a secret key, validated on protected route requests
- ***Role-Based Access Control (RBAC):***
  - Three roles: USER, ADMIN, SUPER_ADMIN
  - Middleware: Verifies roles, prevents unauthorized access to sensitive endpoints
- ***Rate Limiting & Helmet Protection:***
  - Rate limiter: 100 requests per IP every 15 minutes
  - Helmet: Secures HTTP headers with CSP and XSS protection
  - Joi: Validates request bodies before controller logic

MEDIA PROCESSING WORKFLOW

Upload & Compression Pipeline

- *Multer*:
  - Memory storage
  - 5MB limit
  - Accepts JPEG, PNG, WEBP
- *Sharp*:
  - Resize images to 800px width
  - JPEG quality 80%
  - In-memory processing, no disk writes for fast handling

Cloud Storage & CDN Delivery

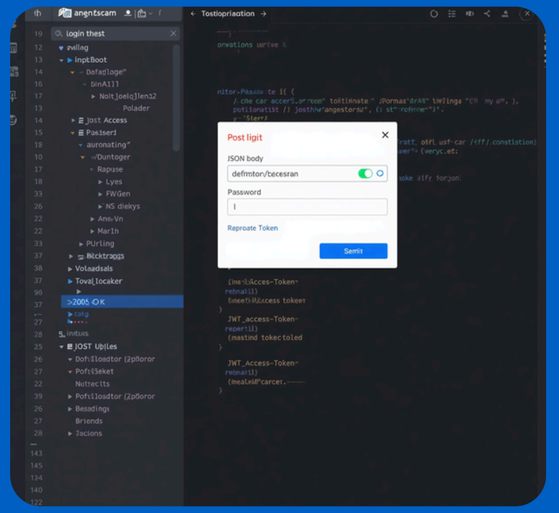
- *Cloudinary*:
  - Receives Sharp-compressed buffer
  - Returns secure URL and public_id
  - URL transformations: optimized delivery (w_500, h_500, c_fill, f_auto, q_auto)
  - Old assets removed on update/delete to avoid orphaned files

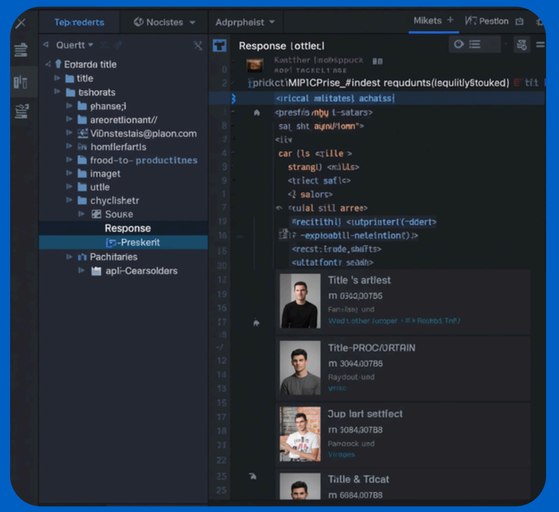
API HIGHLIGHTS

- POST /api/auth/signup – Register a new user account
- POST /api/auth/login – Login and receive JWT access & refresh tokens
- GET /api/products/search?keyword=shirt – Search products by keyword with filters

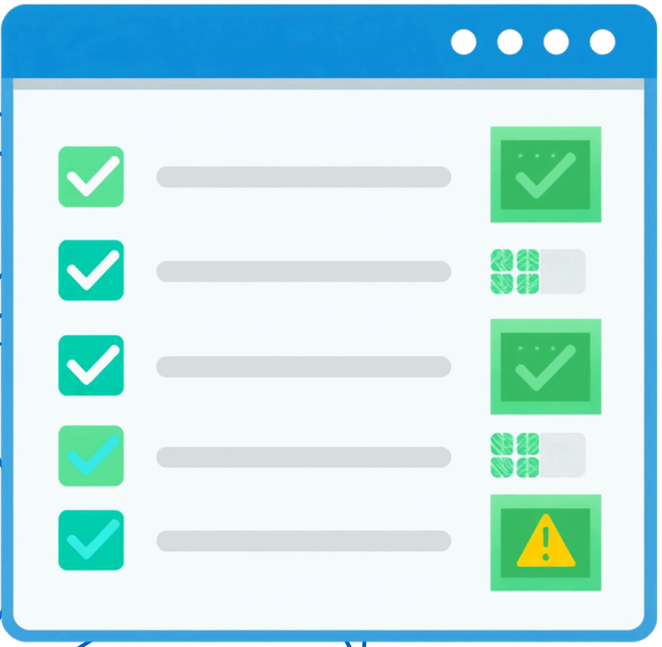
- POST /api/cart – Add item to cart with quantity
- POST /api/orders – Create a new order with shipping address
- POST /api/wishlist/add – Save a product to user wishlist

- POST /api/measurements – Save user body dimensions for size recommendations
- POST /api/tryOnRoutes/upload/tryon – Upload try-on images and store history
- GET /api/products/:id – Retrieve full product details with images





TESTING & RESULTS



T***ools & Scope:***

- Postman and MongoDB Compass for testing
- Modules tested:
  - Authentication
  - CRUD endpoints
  - Request validation
  - Protected routes
  - Centralized error handling
  - Rate limiting
  - Across 12+ developed modules

R***esults:***

- All test cases passed successfully
  - Modules: authentication, product, cart, order, wardrobe, measurements, try-on
- Known issue:
  - Wishlist routes missing authentication middleware
  - Identified and flagged for future fix

AI INTEGRATION PLANNING

AI Avatar Generation: From user photo uploads, generate personalized 3D avatars. Enables realistic virtual try-on experiences and powers future recommendation engines.

Personalized Recommendations: Leveraging ProductMapping metadata combined with stored body measurements to deliver accurate, AI-driven product and size suggestions.

Virtual Try-On AI: Replace mock endpoints with VITON-HD / HR-VITON models. Real-time status updates delivered via Socket.IO (202 Accepted with sessionId on initiation).

SCALABILITY RESEARCH

Redis Caching
Cache product catalog and session data for faster response times. Reduces database load significantly.

BullMQ / RabbitMQ
Async task queue for AI processing jobs. Decouples heavy workloads from the main API thread for reliability.

Nginx Load Balancer
Distributes incoming traffic across multiple Node.js instances. Ensures high availability and fault tolerance.

Docker
Containerizes the backend for consistent, portable deployment across environments with isolated dependencies.

Kubernetes
Orchestrates containers with auto-scaling policies. Manages rolling updates and self-healing deployments.

MongoDB Atlas Sharding
Horizontal database scaling across shards. Supports growing data volumes for users, products, and try-on records.

CHALLENGES & KEY LEARNINGS

Challenges:

- Designing modular and scalable backend architecture from scratch
- Managing complex relationships across 10+ MongoDB collections using Mongoose populate
- Implementing secure JWT authentication with role-based access control (USER, ADMIN, SUPER_ADMIN)
- Planning AI-ready workflows (avatar generation, try-on) without disrupting existing codebase

Key Learnings:

- Advanced Node.js and Express.js architectural patterns for production-grade APIs
- MongoDB indexing, schema design, and population techniques for performance
- JWT and Google OAuth integration strategies with refresh token management
- Efficient image pipeline using Sharp and Cloudinary
- Team collaboration, code reviews, and version control best practices with Git and GitHub

TEAM CONTRIBUTIONS

- Nandini — Integration coordinator; led cart, order, measurement & avatar modules. Vagdevi — Handled authentication, JWT, RBAC, profile & wardrobe modules.
- Bharath (Me) — Wishlist module, API validation, security enhancements, testing & avatar support. Vishnu — Wishlist, try-on support & testing workflows.
- Hrishi Teja — Product catalog, order management & try-on workflows. Nainisha — Profile management, wardrobe module & user preferences.

MY CONTRIBUTIONS

Wishlist Module

- Developed full CRUD operations
- Added duplicate prevention logic
- Used Mongoose .populate() for product detail display

API Validation

- Created Joi validation schemas
- Validated request bodies pre-controller to reduce errors

Security & Testing

- Set rate limiting: 100 requests/15 min
- Configured Helmet headers and role-based middleware
- Performed Postman tests for auth, wishlist, and edge cases

THANK YOU!

Bharath Kumar Nersu | Backend Developer | Raritone Private Limited